

Python for Research

Today: understand what Python is FOR before we worry about code.

Imagine you have 5,000 survey answers

Question: “What do audiences like about sports coverage?”

By hand

Read every answer.
Count repeated ideas.
Keep track of everything.

With Python

Give the computer a clear
set of steps.
Let it repeat those steps.

Your job

Decide the question.
Check whether the steps
make sense.
Interpret the result.

Python does not replace the researcher. It helps with repeated work.

Python is just a way to give instructions

You already give instructions all day:

Coffee order

“Iced latte, oat milk, no syrup.”

Google Maps

“Take me to campus.”

Python

“Count these responses.”

The difference: computers need instructions to be very exact.

A research example you can see

Research question: Which headline words appear most often?

1. DATA

500 news headlines

2. INSTRUCTIONS

Make text lowercase.
Split into words.
Count the words.

3. RESULT

A list of common words

That set of instructions can be written in Python.

When is Python useful?

You have MANY things to process — 1,000 survey responses, posts, or articles.

You need to do the SAME thing many times.

You want a process that can be saved and repeated.

You want to use tools for statistics, text analysis, or AI.

10 interviews? You may read them yourself. 100,000 posts? A computer can help.

Python and Google Colab are NOT the same thing

Python = the language

The instructions you write for the computer.

Google Colab = the notebook

A website where you can write and run Python.

English : Google Docs = Python : Google Colab

What will we actually do in Colab?

WRITE

Put an instruction in a code cell.

RUN

Click ► to ask the computer to do it.

LOOK

Read the output. Ask: does this make sense?

That is the basic cycle: WRITE → RUN → LOOK.

Before code: a “variable” is just a label

Think about labeling a box so you know what is inside.

Box label

favorite_team

What is inside

"Gamecocks"

In Python, we connect them with =

Now the code makes sense

```
favorite_team = "Gamecocks"
```

Read it in plain English:

“I am giving the name `favorite_team` to the text `Gamecocks`.”

Text and numbers are different

TEXT

"Gamecocks"
"I trust this news source"
"sports journalism"

NUMBERS

25
4.7
1000

Quotation marks tell Python: “this is text.”

A function means: DO something

You already understand functions in everyday life.

Calculator

`average(numbers)`

Phone

`call(Mom)`

Python

`print(message)`

`function(thing) ≈ “do this action using this thing”`

Your first tiny Python function

Two steps

```
message = "Hello, JOUR 701"  
print(message)
```

Step 1: Save some text and call it message.

Step 2: Ask Python to show message.

Python can also be a calculator

Suppose 4 students rated an article: 4, 5, 3, 4.

```
(4 + 5 + 3 + 4) / 4
```

OUTPUT

4.0

The code is not the research question. The code is one tool for answering it.

A list is just a group of things

Everyday example: a playlist is a list of songs.

```
headlines = [  
    "Gamecocks win opener",  
    "Coach announces lineup",  
    "Fans react online"  
]
```

[] means: “these things belong together.”

A dictionary is like a labeled profile card

Article card

Source: ESPN
Topic: Sports
Year: 2026

```
article = {  
    "source": "ESPN",  
    "topic": "Sports",  
    "year": 2026  
}
```

{ } stores labeled information: label → value.

Libraries are toolboxes

Python starts with basic tools. Libraries give it more tools.

pandas

Tables and data

matplotlib

Graphs

statistics

Simple statistics

You do NOT need to memorize every library.

Errors are normal

Human instruction

“Meet me at the library.”
Which library? What time?

Computer instruction

```
print("Hello"
```

Missing a parenthesis → error

An error usually means: the computer needs a clearer instruction.

How to use AI when code feels confusing

Ask: “Explain this code one line at a time in plain English.”

Ask: “What does this error mean?”

Ask: “Show me a simpler example.”

Then run the code and check the output yourself.

Goal: understand enough to judge the code — not memorize everything.

The 4 ideas I want you to remember today

1

Python gives instructions.

2

Variables give information a name.

3

Lists group things together.

4

Run code, look at output, revise.

You are learning to READ and USE code, not become a software engineer.

Next class: use Python to organize many pieces of information

Lists → dictionaries → finding items → repeating work

We will always start with the research problem first.